Exact Periodic Barycentric Placement Specification

Stage 7R: Reusable Topology-Derived Rational Placement

mdstats

2026-07-18

Contents

Purpose and boundary

`periodic_barycentric.py` computes and persists the exact rational equilibrium placement of one immutable `PeriodicNetView`. It extracts the placement machinery previously private to automatic symmetry discovery so that later embedding, symmetry, and certification modules can share one authoritative result.

Runtime/API target:

`mdstats 0.19.19a0`

The module supplies topology-derived fractional coordinates only. It does not choose a Euclidean lattice metric, Cartesian cell, natural tiling, face triangulation, or molecular-dynamics frame.

Motivation

The periodic-net discovery backend requires an exact placement whose affine symmetries can be enumerated without floating-point tolerance. Stage 8A will also need a reproducible topology-derived reference realization. Keeping the exact linear solve private inside `net_symmetry_discovery.py` would duplicate theory, resource policy, and serialization.

The extracted object establishes the boundary

```
PeriodicNetView
    -> PeriodicBarycentricPlacement
    -> symmetry discovery / later embedding
```

and prevents discovery records from carrying an anonymous coordinate tuple with no independent source identity.

Algorithmic provenance

The barycentric or equilibrium placement follows Delgado-Friedrichs [1]. The periodic quotient representation is the vector method of Chung, Hahn, and Klee [2]. `mdstats` adds:

- explicit `PeriodicNetView.digest` ownership;
- a deterministic translation gauge;
- exact rational Gaussian elimination;
- rational-coefficient growth limits;

- collision diagnostics modulo $\mathbb{Z}^3$; and
- source-validated deterministic serialization.

Mathematical model

Let the quotient vertices be $i = 0, \ldots, n-1$. For every oriented quotient edge

$$(i, \mathbf{0}) \to (j, \boldsymbol{\delta}_e),$$

the equilibrium condition at vertex i is

$$\sum_{e \ni i} (\mathbf{x}_j + \boldsymbol{\delta}_e - \mathbf{x}_i) = \mathbf{0}.$$

Collecting all vertices gives a graph-Laplacian system

$$LX = B,$$

where $X \in \mathbb{Q}^{n \times 3}$. Translation makes L singular. The module fixes one source atom i_0 and imposes

$$\mathbf{x}_{i_0} = \mathbf{0}.$$

The reduced system is solved exactly over `fractions.Fraction`.

Self-image edges have two opposite incidences at one quotient vertex and make no net contribution to the equilibrium equation.

Collision diagnostic

Two quotient vertices collide in the lifted placement when

$$\mathbf{x}_i - \mathbf{x}_j \in \mathbb{Z}^3.$$

The object records the ordered source-atom pairs satisfying this condition. Collision is diagnostic, not silently discarded. Automatic symmetry discovery requires `collision_free=True`; other consumers may inspect a collided placement without treating it as a stable embedding.

Public API

```python
@dataclass(frozen=True, slots=True)
class PeriodicBarycentricResources:
    max_vertices: int = 4096
    max_fraction_bits: int = 4096
```

`max_fraction_bits` bounds the numerator or denominator bit length observed through exact elimination. Exceeding either resource raises a transactional `PeriodicBarycentricResourceError`; no partial placement is returned.

```python
@dataclass(frozen=True, slots=True)
class PeriodicBarycentricPlacement:
    periodic_net_view_digest: str
    topology_graph_digest: str
    anchor_atom_index: int
    vertex_atom_indices: tuple[int, ...]
    coordinates: tuple[tuple[Fraction, Fraction, Fraction], ...]
    collision_atom_pairs: tuple[tuple[int, int], ...]
    max_fraction_bits_observed: int
    digest: str
```

Builder:

```python
build_periodic_barycentric_placement(
    view: PeriodicNetView,
    *,
    anchor_atom_index: int | None = None,
```

```
    resources: PeriodicBarycentricResources | None = None,
) -> PeriodicBarycentricPlacement
```

The default anchor is the smallest source framework atom index.

Input constraints

The first backend requires:

1. one `PeriodicNetView` quotient component;
2. nonempty, source-stable vertex and edge sets;
3. an anchor belonging to the view; and
4. a nonsingular reduced quotient Laplacian after gauge fixing.

Three-periodicity is not required merely to construct the placement. The exact symmetry-discovery backend imposes its stronger rank-three/index-one conditions separately.

Serialization

Schema:

`mdstats.periodic-barycentric-placement.v1`

Every rational number is serialized as

`[numerator, denominator]`

`from_dict(..., view=...)` rebuilds the placement from the supplied view and requires byte-for-byte canonical payload equality. A digest alone is not treated as scientific verification.

Complexity

The present exact dense elimination has arithmetic complexity approximately

$$O(n^3)$$

and can exhibit rational coefficient growth. This is appropriate for moderate quotient graphs and exact once-per-net calculations. `max_vertices` and `max_fraction_bits` make the supported domain explicit.

Edge cases

- disconnected quotient: unsupported;
- isolated vertex in a nominally connected view: reduced system becomes singular;
- parallel edges: each contributes independently;
- self-image edge: opposite incidences cancel in the equilibrium equation;
- barycentric collision: returned as diagnostic, rejected by discovery;
- anchor change: coordinates change by a common translation gauge but define the same periodic realization;
- partial periodicity: placement may be constructed, but Stage 6C discovery still rejects non-three-periodic views.

Focused tests

The gate must verify:

- exact diamond-net rational coordinates;
- deterministic anchor gauge;
- collision recording;
- source-validated round-trip serialization;
- vertex and rational-bit resource failure; and
- unchanged automatic symmetry results when discovery consumes the extracted object.

References

[1] O. Delgado-Friedrichs, "Equilibrium placement of periodic graphs and convexity of plane tilings," in *Graph Drawing*, LNCS 2912, 178-189 (2004), doi:10.1007/978-3-540-24595-7_17.

[2] S. J. Chung, Th. Hahn, and W. E. Klee, "Nomenclature and generation of three-periodic nets: the vector method," *Acta Cryst. A* **40**, 42-50 (1984), doi:10.1107/S0108767384000088.